AI Engineering and Claude Architect · Part II, Retrieval

Vector Search, Lesson 1

Choosing and Operating Embedding Models

Your search returned confident, well-formatted, completely wrong results for a week. No errors appeared and no alert fired. A deployment had upgraded the embedding model for queries only, and the documents were still in the old space.

Module	AI04 of 13
Lesson	1 of 5
Prerequisites	AI01 L4, AI02 L2, HLD M21A (embeddings and "near")
Exam relevance	Out of scope. Embedding models and vector database details are explicitly excluded
Reading time	~30 minutes

Contents

1. Choosing and Operating Embedding Models

Lesson 1 of 5 · Vector Databases and Search

Choosing and Operating Embedding Models

Key idea

An embedding model defines a private coordinate system, so your index is permanently tied to the exact model that built it. Choose on your own data, using measured retrieval quality, and not a leaderboard. Then operate it like a schema: **record the model version with every vector, refuse mismatched queries in code, and migrate by building a new index beside the old one.** Dimensions and quantization are cost levers, and each one has a recall price you must measure.

> *Before: HLD M21A teaches what embeddings are, why normalization matters, and the "build v2 beside v1" migration. This lesson doesn't repeat them. Teaches: how to choose a model, the query and document distinction, dimension trade-offs, version guards, and embedding pipeline operations. Used by: AI04 L2–L5, AI05, AI08 L5.*

The hook: the failure with no symptoms

A team upgrades its embedding library. The query path picks up the new model immediately. Documents were embedded months ago with the old one.

Real output from `code/ai04-vector/EmbeddingOps.java`, where a second "model" is simulated by shuffling dimensions, so the vectors live in a different space:

```
Query from model A against documents from model B: recall@10=0.000
Chance level for top 10 of 10,000: 0.001
```

Key idea

Recall drops to nothing, and the system keeps returning ten results with plausible-looking scores. Nothing fails. Cosine similarity will happily compare any two vectors of the same length. HLD M21A calls this silent nonsense, and this lesson builds the guard that stops it.

The simulation is an extreme case, and real model pairs vary. The conclusion doesn't change: distances across models are meaningless.

Intuition: two maps with different grids

Two city maps each use a grid. On one, grid square C4 is the train station. On the other, C4 is a lake. If you take a coordinate from one and look it up on the other, you get a valid location that means nothing.

An embedding model is one such map. It has no universal coordinates, so a vector is only meaningful alongside the name of the map that produced it.

Choosing a model

What actually matters

Criterion	Why it matters	How to judge it
Retrieval quality on your data	The only thing that really counts	Recall@k on a labeled set (AI04 L3)
Domain fit	Legal, code and finance vocabulary differ	Compare a domain model with a general one
Context length	Limits your chunk size	Check the model's token limit
Dimensions	Drive storage and memory	See below
Languages	Mixed-language corpora need support	Test queries in each language
Latency and throughput	Query embedding sits on your hot path	Measure p99 from your region
Data handling	Your text leaves your system	Check residency and retention
Stability	Model retirement forces a full re-embed	Check lifecycle policies

A public leaderboard is a weak guide, just as in AI02 L7. It measures someone else's corpus.

What's on offer

Anthropic doesn't offer its own embedding model. Its guide covers Voyage AI, while advising you to assess a variety of vendors.

Voyage model	Stated use	Context	Dimensions
`voyage-4-large`	Best general-purpose and multilingual quality	32,000	1024 by default, or 256, 512, 2048
`voyage-4`	Balances quality and efficiency	32,000	Same
`voyage-4-lite`	Latency and cost	32,000	Same
`voyage-code-4`	Code retrieval and agentic coding	32,000	Same
`voyage-context-4`	Chunk embeddings that capture document context	120,000 with auto-chunking	Same
`voyage-law-2`, `voyage-finance-2`	Legal or finance	16,000, 32,000	1024

These details were read from that guide in November 2026. Other options include OpenAI's and Cohere's models, and open models you host yourself, such as those in the BGE family. Test before you commit.

Query or document?

Many modern models treat these differently. Voyage's `input_type` prepends a different prompt to each, and the guide says not to omit it for retrieval:

`input_type`	Prompt added
`query`	"Represent the query for retrieving supporting documents: "
`document`	"Represent the document for retrieval: "

This is a second, subtler version of the mismatch bug: documents embedded as `query`, or both embedded with no type, can lower quality with no error.

Dimensions: the cost lever

Some models support **Matryoshka** embeddings, built so that shorter prefixes are still useful. You can ask for shorter vectors from the API, or truncate stored ones. Truncated vectors **must be normalized again**.

Real output, on synthetic vectors whose early dimensions carry more variance, to mimic that property. **This isn't a real embedding model.** The curve's shape is the lesson, not its values.

```
Shortening vectors (synthetic data, recall against full length)
  dim=1024  recall@10=1.000 size=100.0% of full
  dim=512   recall@10=0.983 size= 50.0% of full
  dim=256   recall@10=0.953 size= 25.0% of full
  dim=128   recall@10=0.911 size= 12.5% of full
  dim=64    recall@10=0.853 size=  6.3% of full
```

```
static float[][] truncate(float[][] vectors, int dim) {
    float[][] out = new float[vectors.length][];
    for (int i = 0; i < vectors.length; i++) {
        // Shortened vectors must be normalized again.
        out[i] = normalize(Arrays.copyOf(vectors[i], dim));
    }
    return out;
}
```

And what it costs, computed for 100 million chunks:

```
  dim=256   float32=    102 GB  int8=     26 GB  binary=     3 GB
  dim=512   float32=    205 GB  int8=     51 GB  binary=     6 GB
  dim=1024  float32=    410 GB  int8=    102 GB  binary=    13 GB
  dim=2048  float32=    819 GB  int8=    205 GB  binary=    26 GB
```

Halving dimensions halves memory, and it also speeds up each distance calculation. Run this sweep on your own data, since the curve depends entirely on the corpus. AI04 L2 covers quantization, the other cost lever.

Operating embeddings

Rule 1: Version every vector

Store the model name, version, dimensions and input settings with each index. Then check them at query time.

```
public static void requireSameModel(String queryModel, String indexModel) {
    if (!queryModel.equals(indexModel)) {
        throw new IllegalStateException(
                "Query model " + queryModel + " doesn't match index model "
                        + indexModel);
    }
}
```

A failing query is far better than a confidently wrong one. Here's the path in Java, from `code/ai04-vector/pgvector/.../EmbeddingClient.java`. It compiles, and hasn't been run, since no Voyage key is available.

```
public List<Embedding> embed(List<String> texts, InputType type)
        throws IOException, InterruptedException {
    String body = json.writeValueAsString(Map.of(
            "input", texts,
            "model", model,
            "input_type", type.name().toLowerCase()));
    HttpRequest request = HttpRequest.newBuilder(ENDPOINT)
            .timeout(Duration.ofSeconds(30))
            .header("Content-Type", "application/json")
            .header("Authorization", "Bearer " + apiKey)
            .POST(HttpRequest.BodyPublishers.ofString(body))
            .build();
```

The `InputType` enum makes the query and document distinction impossible to forget, and the returned `Embedding` record carries its model version with it.

Rule 2: Embed in batches, with idempotent writes

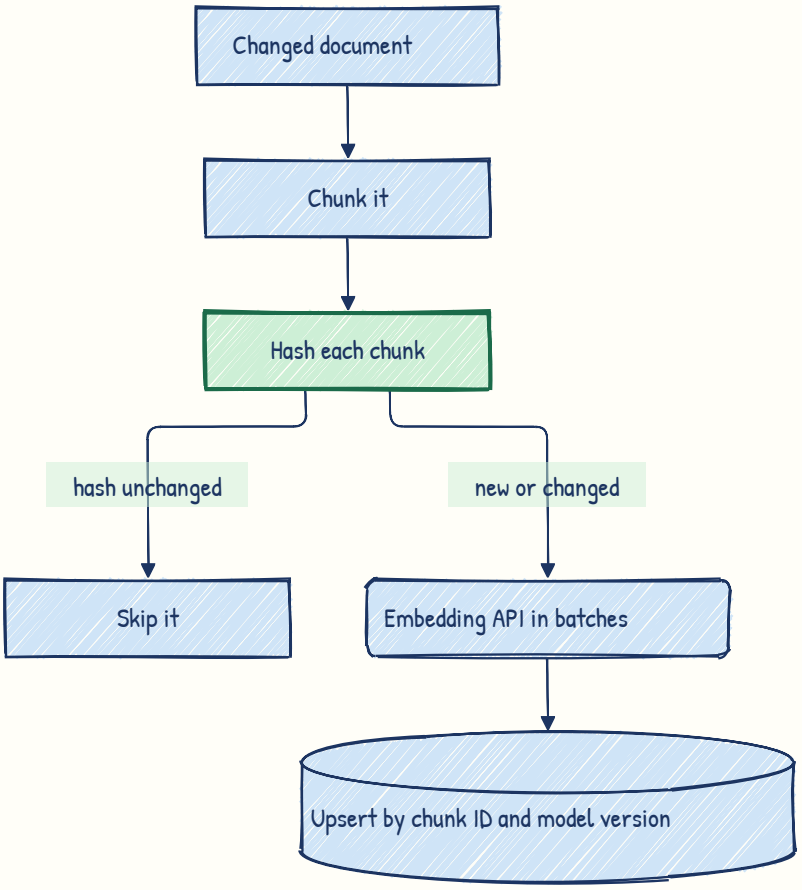


Figure 1. An embedding pipeline. Hashing the text avoids paying to re-embed unchanged chunks, and the unique key makes retries harmless.

Practice	Why
Hash chunk text	Skip unchanged chunks, and save money on re-indexing
Batch requests	Better throughput, within provider limits
Upsert by `(chunk_id, model_version)`	Retries can't duplicate, and two versions can coexist
Back off on rate limits	Ingestion mustn't starve live query embedding
Separate credentials or quotas for live and bulk work	Protects query latency

Rule 3: Cache query embeddings

Popular queries repeat. Key the cache on `(model_version, input_type, normalized_query_text)`. **Leaving the version out of the key creates the hook's bug through a cache**, right after a migration.

Rule 4: Migrate beside, then switch

HLD M21A explains why it must be `v2` beside `v1`. Here's the operational checklist:

Step	Check
1. Evaluate the candidate on your labeled set	Is the gain worth a migration?
2. Build `v2` from stored chunk text	Keep the raw text, or you can't re-embed
3. Double-write new chunks to both	`v2` stays current during the build
4. Compare recall on real queries	Decide using evidence
5. Switch queries by a flag, gradually	Rollback is one flag
6. Keep `v1` until confident, then drop it	Avoids a second migration under pressure

Why the obvious choices fall short

Choice	Why it falls short
Pick the top leaderboard model	Not your data
Largest dimensions by default	Costs memory, and may add nothing measurable
Store vectors only	You can't re-embed, and a prompt needs the words
Upgrade the model in place	No consistent moment to serve from
Trust the library's default model	A dependency bump can silently change it, so pin it

How the field and Anthropic approach it

Problem	Approach
Asymmetry between short queries and long documents	`input_type`, or asymmetric models
Cost of large vectors	Matryoshka dimensions and quantized output types
Chunks losing document context	Contextualized models such as `voyage-context-4`, or Anthropic's Contextual Retrieval (AI05 L3)
Quality varies by corpus	Domain-specific models

Anthropic's part is small but useful: it publishes guidance and Contextual Retrieval, and doesn't sell an embedding model. So a RAG system on Claude normally has **two providers**, with two sets of limits and failure modes.

The production architecture

This is a recommended reference design, not a specific company's system.

Baseline: embed inline, with an unpinned model. It's simple, and exposed to the hook's bug.

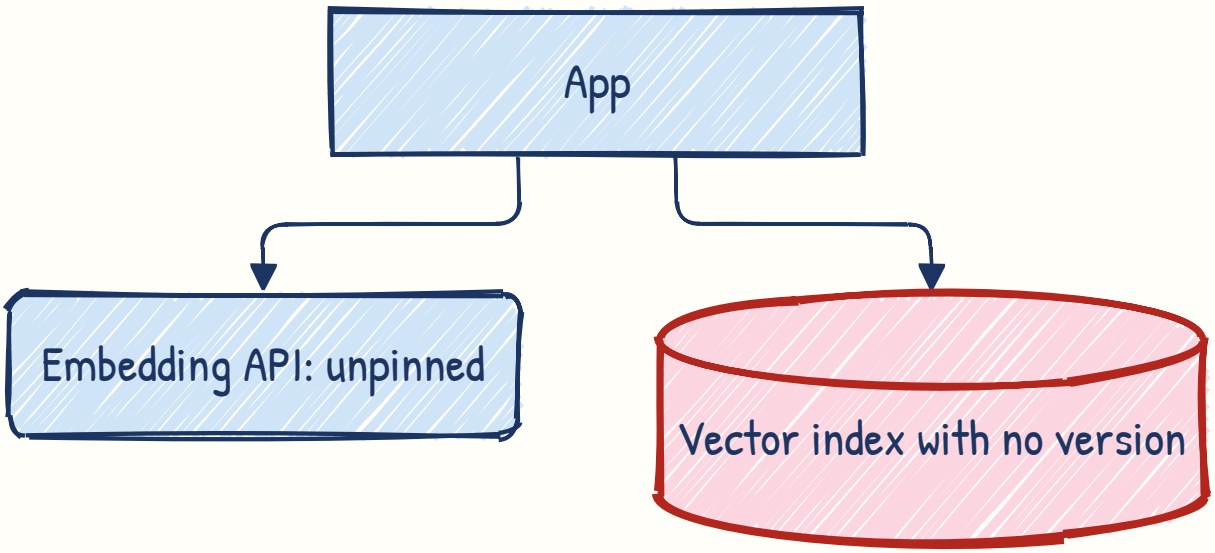


Figure 2. Baseline. The model is whatever the library defaults to, and nothing records which model built the index.

Improved: pin and version the model. Mismatches now fail loudly. New problem: **query embedding is now a critical dependency on your hot path**, and a provider outage stops all search.

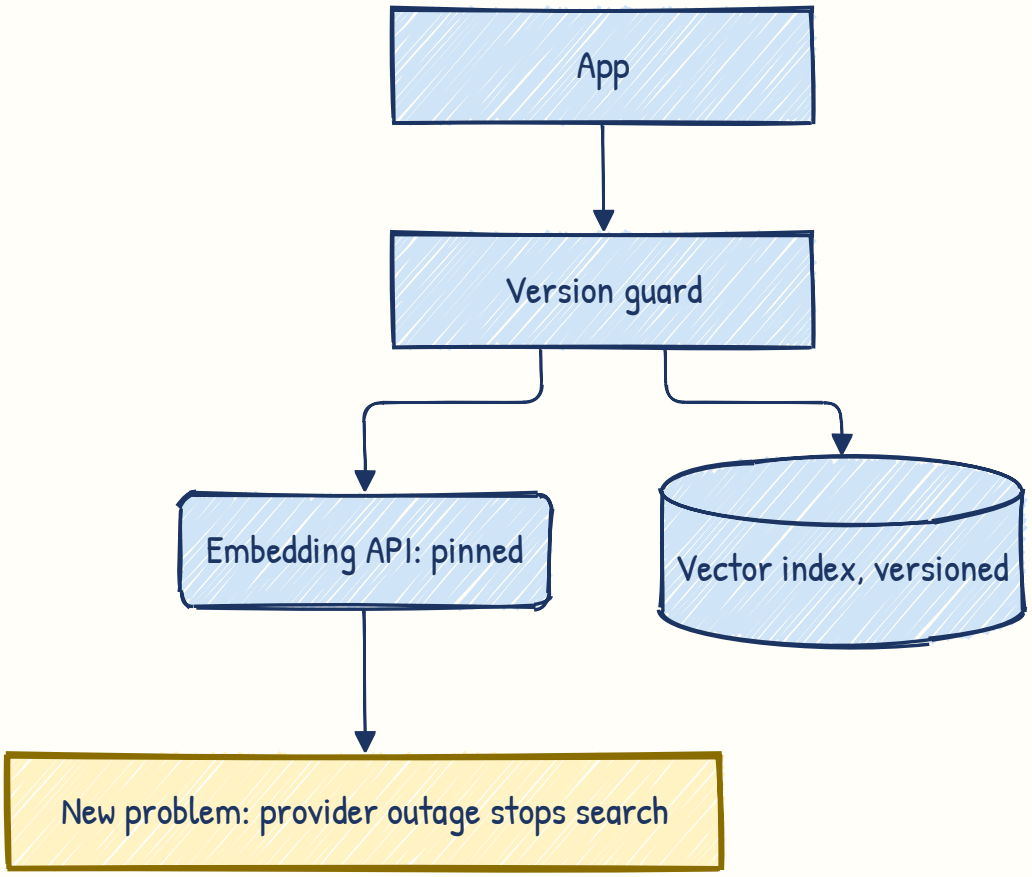


Figure 3. Improved. Versioning prevents silent nonsense, but the embedding provider can now take search down.

Production: cached, with a degraded mode, and separate ingestion.

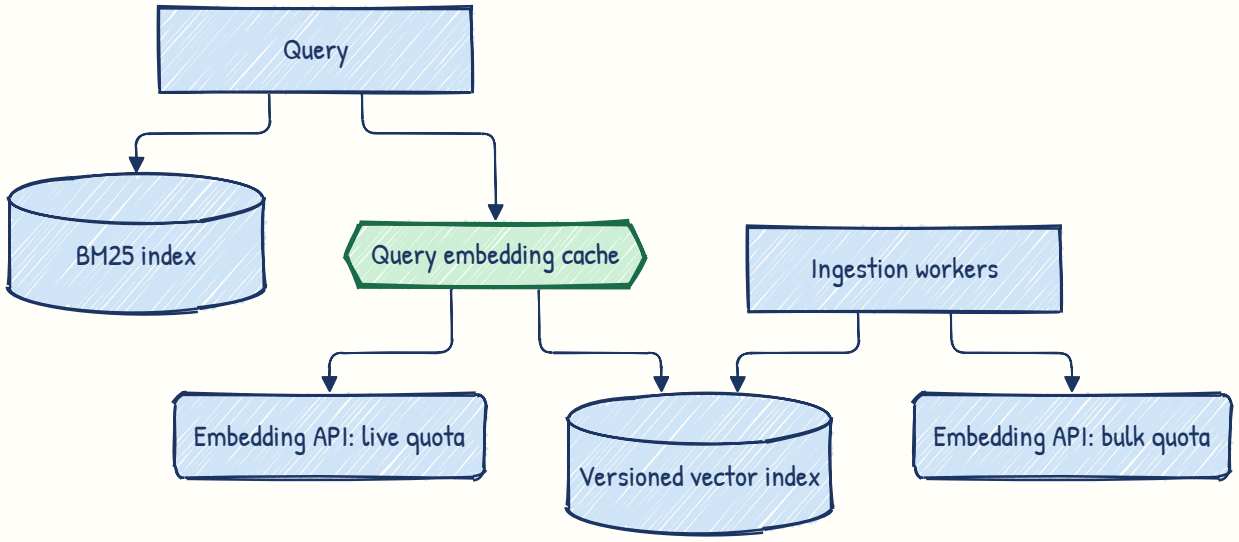


Figure 4. Production. Live queries use a cache and fall back to keyword search during an outage. Ingestion runs separately, with its own quota.

Component	Why it exists	How it communicates
Query cache	Cuts latency and cost on repeats	Redis, keyed with the model version
Live and bulk quotas	Bulk work mustn't starve queries	Separate keys or a local rate limiter
BM25 index	Search still works if embedding fails	Queried in parallel (AI04 L3)
Version guard	Stops mismatches	In-process check

Failures

Failure	Effect	Containment
Embedding provider down	No new query vectors	Serve cached vectors, and fall back to BM25, flagged as degraded
Provider retires your model	Forced migration	Track lifecycle dates, and keep raw text
Wrong `input_type`	Quality loss, no errors	Typed API, and evals
Partial re-embed	Mixed space	Never mix. Use a new index
Cache keyed without version	Old vectors served after a switch	Include the version in the key

Trade-offs

Decision	Gain	Cost
Larger dimensions	Possibly better recall	Memory and latency
Domain model	Better on specialized text	Narrow use, and possibly less maintained
Self-hosted model	Data control, no per-call cost	You run GPUs (AI11 L2)
Query cache	Lower latency	Invalidation on migration
Keeping raw text	Can always re-embed	Storage

When not to bother tuning: for a small corpus, a good default model and an eval are enough. Under about 200,000 tokens, you may not need embeddings at all (AI05 L1).

Common mistakes

1. **Comparing vectors from different models.** There's no error, and the results are meaningless.
2. **Ignoring `input_type`.** It's a quiet quality loss.
3. **Forgetting to re-normalize truncated vectors.** Dot-product ranking then misbehaves.
4. **Not storing the chunk text.** You can't re-embed or build a prompt.
5. **Letting bulk jobs share a quota with live queries.** Ingestion causes search latency.
6. **Choosing dimensions without a sweep.** It's the cheapest experiment you can run.

What to say in an interview

"I'd treat the embedding model like a schema version: pinned, stored with every vector, and checked at query time, because a mismatch returns confident nonsense with no error. I'd pick the model using recall on our own labeled data, then sweep dimensions, since halving them halves memory. Migrations build a second index, and BM25 keeps search alive if the provider goes down."

Certification lens

How this lesson maps to CCAR-F

Not tested. Embedding models and vector database implementation details are explicitly out of scope. Skim this module if your exam is close, apart from the provenance ideas in AI05.

These are original practice questions for understanding, not exam simulations.

Q Q1 · Troubleshooting · Medium. After a deployment, search quality collapses, with no errors, and scores look normal. Name the first thing you'd check, and the guard that would have prevented it.

A Whether queries and documents use the same embedding model and version. An unpinned dependency can change the model silently. Pin the model, store its version with the index, and refuse mismatched queries. Also check that any query cache key includes the version.

Q Q2 · Calculation · Medium. You have 40 million chunks at 1024 dimensions in float32. What's the raw vector size? And at 512 dimensions in int8?

A 40M × 1024 × 4 B = **163.84 GB**. 40M × 512 × 1 B = **20.48 GB**, one-eighth of the size. Both figures exclude the index graph and metadata.

Q Q3 · Best decision · Hard. A vendor releases a new model with 8% better benchmark scores. Your team wants to migrate next sprint. What do you do?

A Test it on your labeled set first, since a benchmark isn't your corpus. If it wins by enough to justify the cost, build a `v2` index beside `v1` from stored text, double-write, compare on real queries, then switch by flag. If it doesn't, don't migrate.

Revision - Lesson 1

→ An embedding model defines a private space, so an index is tied to the model that built it.

→ A mismatch gives confident nonsense with no error. Recall was 0.000 in the demo.

→ Choose using recall on your own labeled data, not a leaderboard.

→ Anthropic offers no embedding model. Its guide covers Voyage AI, and suggests assessing vendors.

→ input_type separates queries from documents. Use it.

→ Shorter vectors save memory and time. Re-normalize after truncating, and measure recall.

→ Version every vector and refuse mismatches in code.

→ Hash chunks to avoid re-embedding, and upsert by chunk ID and model version.

→ Include the model version in query cache keys.

→ Keep raw chunk text, separate bulk and live quotas, and fall back to BM25 in an outage.

Your notes